OpenZeppelin | security

1inch Aqua and SwapVM MVP v1.0 Audit

April 9, 2026

Table of Contents

Summary

Type	DeFi
Timeline	From 2026-01-12 To 2026-02-02
Languages	Solidity

Total Issues	53 (33 resolved)
Critical Severity Issues	3 (3 resolved)
High Severity Issues	2 (2 resolved)
Medium Severity Issues	12 (6 resolved)
Low Severity Issues	16 (6 resolved)
Notes & Additional Information	20 (16 resolved)

Scope

OpenZeppelin performed an audit of the following repositories:

- The 1inch/swap-vm repository at commit 60172b
- The 1inch/aqua repository at commit 89cedfa
- The 1inch/solidity-utils repository at commit 38e5f8d.
- During fix review, we conducted audit of the new refactored `XYCConcentrate` instruction at commit 9b9d821.
- Additionally commits af53fc3 on `Aqua` and f6631a0 on `SwapVM` were reviewed implementing the `Rescuable` contract introduced in `solidity-utils` at commit 81ad207 making them the final commits reviewed for `Aqua` and `solidity-utils` during the engagement.
- The final `SwapVM` commit reviewed for the engagement was b2daef8. The final reviewed `SwapVM` state includes a maker-favoring rounding adjustment.

In scope were the following files:

```
1inch/swap-vm
├── src
│   ├── instructions
│   │   ├── Balances.sol
│   │   ├── Controls.sol
│   │   ├── Decay.sol
│   │   ├── Extruction.sol
│   │   ├── Fee.sol
│   │   ├── PeggedSwap.sol
│   │   ├── XYCConcentrate.sol
│   │   └── XYCSwap.sol
│   ├── libs
│   │   ├── MakerTraits.sol
│   │   ├── PeggedSwapMath.sol
│   │   ├── Power.sol
│   │   ├── TakerTraits.sol
│   │   └── VM.sol
│   ├── opcodes
│   │   └── AquaOpcodes.sol
│   ├── routers
│   │   └── AquaSwapVMRouter.sol
│   └── SwapVM.sol

1inch/aqua
├── src
│   ├── Aqua.sol
```

```
│   ├── AquaApp.sol
│   ├── AquaRouter.sol
│   ├── interfaces
│   │   └── IAqua.sol
│   └── libs
│       └── Balance.sol

1inch/solidity-utils
├── contracts
│   ├── libraries
│   │   ├── Calldata.sol
│   │   ├── CalldataPtr.sol
│   │   ├── Transient.sol
│   │   └── TransientLock.sol
│   └── mixins
│       ├── Multicall.sol
│       └── Simulator.sol
```

System Overview

Aqua

Aqua is a decentralized shared liquidity layer designed by 1inch to address capital inefficiencies within the DeFi ecosystem. Traditional AMMs (Automated Market Makers) often require liquidity providers to "lock" assets into specific pools, which leads to fragmented and underutilized capital. Aqua introduces a model where liquidity remains in the user wallet while being available for multiple applications to utilize according to market demand.

The protocol functions as an accounting and fund-management engine that sits between liquidity providers and trading applications. Liquidity providers, referred to as makers, grant token approvals to the Aqua contract instead of depositing them into a traditional pool. This allows makers to maintain self-custody of their assets while the protocol manages the permissions and balances required for automated execution.

Makers define their participation through a mechanism called "shipping a strategy". When a maker ships a strategy, they provide a custom calldata that defines the rules of the engagement and the specific tokens involved. This strategy is identified by a unique hash and becomes immutable once active. This design ensures that the rules governing the liquidity cannot be changed unexpectedly by any party during the lifecycle of the strategy.

Applications, or AquaApps, are the external contracts that implement the actual trading logic, such as constant product formulas or limit order books. These applications interact with the core `Aqua` contract to move funds. When a trade is executed, the application pulls the necessary tokens from the maker and subsequently verifies that the taker has pushed the required counterparty tokens back into the maker's balance.

The core `Aqua` contract is responsible for updating internal balances and emitting events that reflect the movement of assets. It does not enforce specific pricing logic but acts as a secure conduit for transfers. By separating the liquidity management from the application logic, the protocol allows for a highly flexible environment where the same capital can support various trading strategies simultaneously.

Swap VM

SwapVM is a programmable execution engine that provides a highly customizable framework for executing swaps. It functions as a virtual machine where the logic of a trade is not hardcoded but defined by a series of instructions. This flexibility allows makers to create sophisticated order types that can include custom fee structures, price calculations, and specific execution constraints.

At the heart of the system is the concept of a program, which is a piece of bytecode containing chained instructions. Each **instruction** corresponds to an opcode implemented by the application contract, such as a constant product formula or a decay mechanism or fee mechanism. When a trade occurs, the VM loops through these instructions to compute the final input and output amounts required for the swap. An opcode is the index of the internal function selector in the `instructions` array. Thus, SwapVM provides the ability for makers to create dynamic programs that can loop through the predefined set of internal functions. However, unlike multicall, these instructions have the ability to jump to the next instructions in-between, manipulate program bytes, and affect the outcome of the swap for the maker.

At its core, the VM maintains four internal registers (`balanceIn`, `balanceOut`, `amountIn`, and `amountOut`) in the `SwapRegisters` struct to track token balances and swap amounts throughout the execution loop, along with a program counter (`nextPC`). Each instruction can update these registers or the program counter based on its functionality. At the end of the loop, the resulting `amountIn` is fulfilled by the `taker` in `tokenIn`, and `amountOut` is fulfilled by the `maker` in `tokenOut`.

A maker - considered as a liquidity provider - initiates the process by constructing an order that includes their specific program and operational traits. These traits, managed by the `MakerTraits` library, define whether the order uses Aqua for liquidity or relies on off-chain EIP-712 signatures for validation. This dual approach allows the SwapVM to operate both as a standalone settlement layer and as a specialized application on top of the Aqua protocol.

A taker initiates the swap using the exact order data signed or shipped (in Aqua) by the maker, including the required `tokenIn`, `tokenOut`, and either `amountIn` or `amountOut`. The taker must supply the correct program and operational traits specified by the maker in order to generate a valid `orderHash` to proceed. The `TakerTraits` contract provides slippage protection to the taker as per the `thresholdAmount` value set during the swap.

The execution flow within SwapVM is strictly governed by the sequence of opcodes chosen by the maker. Once the program completes and the final amounts are determined, the contract handles the transfer of assets between the maker and the taker to finalize the trade. The

`quote` function simulates the swap in a **static context**, allowing callers to verify both the correctness of the program implementation and the expected swap outcome, while the `swap` function performs the actual swap execution. During execution in the static context, instructions are restricted from modifying contract state, and no instruction updates its associated storage. An ideal program is expected to ensure that both `quote` and `swap` result in the same output and quote/swap consistency is maintained.

SwapVM introduces hooks that allow makers and takers to manage liquidity dynamically during the execution cycle. Through the `preTransferIn`, `postTransferIn`, `preTransferOut`, and `postTransferOut` functions, users can perform targeted actions before and after asset transfers.

Security Model and Trust Assumptions

Throughout the in-scope codebase, the following trust assumptions and privileged roles were identified:

- **Asset Compatibility:** The protocol assumes that fee-on-transfer tokens and rebasing tokens will not be used. Such assets can lead to accounting discrepancies in both SwapVM and Aqua, because the contract expects the amount sent to be exactly what is received.
- **Strategy Uniqueness:** Makers must use a unique salt for every strategy when integrating SwapVM with Aqua. If the same program is used for different token pairs without a unique salt, it generates identical order hashes that could enable takers to swap unauthorized tokens.
- **Application Integrity:** All applications developed on top of Aqua and SwapVM are assumed to be securely implemented and audited. Since these applications have the power to trigger fund movements, their security is vital to the safety of the core protocol.
- **User Awareness:** Protocol participants are assumed to understand the risks associated with the specific applications they choose to use. Users must perform their own security assessments of any third-party logic before committing liquidity or executing trades.
    - **Maker**: The maker is assumed to be a sophisticated user with a strong understanding of the application's instruction set and the ability to construct valid programs correctly. Errors in program construction can lead to unintended or unsafe behavior.

    - **Taker**: The taker is expected to thoroughly review and verify the maker's program before executing a swap. A malicious maker may construct a program that results in loss of funds, and it is the taker's responsibility to detect and avoid such programs.
- **Fees**: The maker is assumed to be aware that `flatFees` accrued are automatically reinvested in the strategy and included in the internal accounting of balances.

Privileged Roles

Aqua

- **Maker:** The maker is the liquidity provider who owns the underlying assets and creates strategies. This role has the exclusive authority to authorize an application to use their tokens through the `ship` function. Additionally, the maker retains the power to terminate a strategy and revoke token access at any time by calling the `dock` function.
- **AquaApp:** Applications are smart contracts authorized by makers to execute trades on their behalf. An application has the privileged capability to `pull` tokens from a maker wallet as long as a valid strategy hash is provided. These contracts are trusted to implement correct pricing logic and to verify that the maker receives the appropriate tokens from the taker.
- **Taker:** The taker is the counterparty in a trade who interacts with an AquaApp to swap assets. While not a privileged role in terms of protocol management, the taker is responsible for `pushing` the required funds to the maker to satisfy the conditions of the strategy. The protocol relies on the application logic to ensure this push occurs before the transaction completes.

SwapVM

- **Maker:** The liquidity provider who creates and authorizes trading orders. The maker is responsible for defining the bytecode program that governs the swap logic and pricing. They also choose the security model for their orders, either by shipping a strategy in Aqua or by providing cryptographic signatures for direct swaps.
- **App/Router:** The smart contract that inherits from SwapVM and implements the available opcodes. This contract acts as the entry point for takers and defines the set of instructions that makers can use in their programs. It is trusted to correctly map instruction IDs to the intended logic and to manage the execution environment.
- **Taker:** The participant who executes an order created by a maker. The taker provides the necessary traits and data to satisfy the order requirements, such as minimum threshold

amounts (slippage) or deadlines. They interact with the App contract to trigger the swap and are responsible for supplying the input tokens required by the maker program.

Critical Severity

C-01 Precision Loss in `solve()` Quadratic Solver Breaks Invariant Protection

The `PeggedSwap` instruction implements a square-root linear curve for pegged asset swaps using the following invariant formula:

$$\sqrt{\frac{x}{X_0}} + \sqrt{\frac{y}{Y_0}} + A\left(\frac{x}{X_0} + \frac{y}{Y_0}\right) = C$$

where `x` and `y` are the pool's current token reserves, X_0 and Y_0 are the normalization factors, `A` (referred to as `linearWidth` in code) controls how linear versus curved the pricing behaves, and `C` is the invariant constant representing the pool's total value. In a correctly functioning swap, `C` should remain constant or slightly increase due to rounding that favors the maker. When `C` decreases, the maker has paid out more tokens than the curve dictates—a direct financial loss.

The protocol's documentation in `PeggedSwap.sol` provides parameter guidance for different use cases:

```
Parameters guide:
  - For stablecoins (USDC/USDT): A ≈ 0.8-1.5
  - For wrapped tokens (WETH/stETH): A ≈ 0.3-0.6
  - For volatile pairs: A ≈ 0.0-0.2
```

The suggested range for volatile pairs (A ≈ 0.0-0.2) includes values near zero, which overlaps with the numerically unstable region described below.

The `PeggedSwapMath.solve()` function rearranges the invariant equation to find `v` (normalized output reserve) given `u` (normalized input reserve). It calculates the right side as $R = C - \sqrt{u} - A \cdot u$, then solves the quadratic equation $A \cdot w^2 + w - R = 0$ using the standard formula $w = \frac{-1+\sqrt{1+4AR}}{2A}$, and finally computes $v = w^2$. The implementation computes the numerator as `sqrtDiscriminant - ONE` (i.e., $\sqrt{D} - 1$) and divides by `2 * a`. When `A` is extremely small, this formula suffers from catastrophic cancellation—a well-known numerical analysis problem where subtracting two nearly equal numbers destroys significant digits. Specifically, when `A` is tiny the discriminant $D = 1 + 4AR/\text{ONE} \approx 1$, so

$\sqrt{D} \approx 1$ and the numerator $\sqrt{D} - 1 \approx 0$, producing an incorrect `v` value regardless of the small denominator.

Consider a balanced pool with 1,000 tokens of each asset:

Step	Formula	Healthy (`A = 0.8 × 10²⁷`)	Vulnerable (`A = 2`)
Initial State		`x₀ = y₀ = 1000e18`	`x₀ = y₀ = 1000e18`
Invariant Before	$C = \sqrt{u} + \sqrt{v} + A(u + v)$	`3,600,000...e27`	`2,000,000...e27`
Swap Input	`Δx = 100e18`	100 tokens	100 tokens
New u	$u_1 = \frac{x_0 + \Delta x}{X_0}$	`1.1e27`	`1.1e27`
Solve for v₁	$v_1 = w^2$ where $w = \frac{-1+\sqrt{1+4AR}}{2A}$	`0.9019...e27`	`0.8984...e27`
Tokens Out	$\Delta y = y_0 - \lceil v_1 \cdot Y_0 \rceil$	98 tokens	~101.5 tokens
Invariant After	$C' = \sqrt{u_1} + \sqrt{v_1} + A(u_1 + v_1)$	`3,600,000...e27 + 88,518`	`1,928,751...e27`
Invariant Change	$\Delta C = C' - C$	+88,518 ✓	-71,248...e27 ✗
Maker Outcome		Protected	Loses 3.56% of pool value

With a healthy `linearWidth`, the maker gives 98 tokens and the invariant increases by 88,518 units, confirming the rounding protects them. With `linearWidth = 2`, the precision errors cause the maker to give out ~101.5 tokens while the invariant decreases by 3.56%—this loss compounds with every subsequent swap against the pool.

In the above example, the impact demonstrated would only affect a small portion of the pool, providing a small profit to the user. However, besides this exploitation being scalable through a looping swap, during fuzzing tests, it was found that it is possible to easily drain more than 15% of the pool in a single swap, depending on the value of `A` and by manipulating the amount of `amountIn` entered.

Consider replacing the unstable quadratic formula with its mathematically equivalent stable form in `PeggedSwapMath.solve()`:

```
- // w = (-1 + √D) / (2a)  — unstable when a is small
- uint256 numerator = sqrtDiscriminant - ONE;
- uint256 denominator = 2 * a;
- uint256 w = (numerator * ONE) / denominator;
+ // w = 2·rightSide / (1 + √D)  — stable for all a values
+ uint256 denominator = ONE + sqrtDiscriminant;
+ uint256 w = (2 * rightSide * ONE) / denominator;
```

Both formulas produce identical results mathematically, but the second form avoids subtracting two numbers that become nearly equal when A is small— mitigating the root cause of the precision loss.

Update: *Resolved in pull request #67.*

C-02 Shared Liquidity Tracking in Multi-Token `XYCConcentrate` Enables Cyclic Arbitrage

The `XYCConcentrate` instruction implements concentrated liquidity, allowing pools to amplify effective token balances within a price range. The mechanism works as follows:

- Virtual balances are computed via `concentratedBalance()`: `balance + (delta * currentLiquidity) / initialLiquidity`
- The `currentLiquidity` value is stored per order in `liquidity[orderHash]`
- After each swap, `_updateScales()` recalculates liquidity as `sqrt(balanceIn * balanceOut)` using only the two tokens involved in that swap

When a pool contains three tokens (A, B, C) under the same `orderHash`, all token pairs read from and write to the same `liquidity[orderHash]` slot. The problem is that each pair's liquidity should be independent—the mathematical relationship between A and B has no bearing on the relationship between B and C.

However, when an A→B swap occurs, it overwrites the shared liquidity with a value derived solely from A and B balances. When a subsequent B→C swap reads this value to compute virtual balances for B and C, it uses a fundamentally incompatible number, causing the swap to execute at a distorted rate. An attacker exploits this by executing a circular trade through all three pairs:

1. Swap A→B, which sets `liquidity[orderHash]` based on A-B balances
2. Swap B→C using the A-B derived liquidity to compute B-C virtual balances, resulting in a favorable rate
3. Swap C→A using the now B-C derived liquidity, again at a distorted rate

4. The cycle completes with more tokens than started

This PoC showcases asset loss when cycling assets.

Consider implementing per-pair liquidity tracking using a nested mapping like `liquidity[orderHash][pairId]`.

Update: *Resolved in pull request #82. The support for multiple token strategies was dropped and `XYCConcentrate` instruction was updated to a stateless model designed to operate strictly within a two-token model for the current release.*

C-03 Invariant Violation and Fund Drainage Due to Axis Mismatch in PeggedSwap

The `_peggedSwapGrowPriceRange2D` function calculates the swap invariant using local variables `x0` and `y0`, which correspond to `balanceIn` and `balanceOut`. However, the normalization logic incorrectly divides these dynamic values by the static configuration parameters `config.x0` and `config.y0`. This mismatch occurs because the configuration parameters are statically mapped to the token sort order, while the local variables depend on the swap direction.

When a swap occurs in the reverse direction (where `tokenIn` > `tokenOut`), the local `x0` represents the balance of the greater token, but it is normalized by the capacity of the lower token (`config.x0`). This effectively swaps the axes of the bonding curve. If the pool has asymmetric capacities, the contract calculates a wildly incorrect invariant, leading to a broken pricing model where the pool offers exchange rates that do not reflect the actual market value.

Assume a pool with **Token A** (Lower Address) and **Token B** (Greater Address) that is asymmetric. For simplicity, assume `linearWidth (A) = 0`:

- **Capacity A:** 1,000,000 (Abundant asset)
- **Capacity B:** 1,000 (Scarce asset)
- **Current State:** Balanced relative to capacity (Balance A = 1M, Balance B = 1k).
- **True Invariant:** sqrt(1) + sqrt(1) = 2

The Attack (Reverse Swap: B -> A):

1. User swaps **Token B** (`tokenIn`).
2. The contract sets local `x0` = Balance B (1,000).
3. The contract incorrectly calculates the normalized input using `config.x0` (Capacity A): *u=1,000/1,000,000=0.001*

4. The contract calculates the target invariant based on this cross-wired math: *K = sqrt(0.001) + sqrt(1000) ~= 31.65*

The contract now believes that the invariant is **31.65** (far higher than the real 2.0). To satisfy this inflated invariant during the swap solving process, the contract will output a massive amount of Token A for a small amount of Token B, effectively draining the pool reserves. This PoC showcases the fund drainage when the swap is reversed.

Consider dynamically assigning the normalization factors based on the swap direction. If `tokenIn > tokenOut`, the calculation should use `config.y0` to normalize the input balance and `config.x0` to normalize the output balance.

Update: *Resolved at PR #68. The fix introduces dynamic assignment of normalization factors based on the swap direction, using the `parseRatesAndBalances` function, effectively mitigating the issue.*

High Severity

H-01 `XYCConcentrate` Records Incorrect Balances When Protocol Fees Are Extracted

`XYCConcentrate` tracks pool liquidity to scale virtual reserves for concentrated liquidity AMMs. After each swap, `_updateScales` records the new liquidity based on `amountIn` and `amountOut`—the tokens that entered and left the pool. Protocol fee instructions like `_aquaProtocolFeeAmountOutXD` temporarily inflate the swap amount, execute the swap, then **restore the original amount** before extracting the fee. When `XYCConcentrate` wraps a protocol fee instruction, `_updateScales` runs **after** restoration—recording the original amount instead of the actual tokens transferred.

The `AquaAMM` strategy places `XYCConcentrate` before `_aquaProtocolFeeAmountOutXD`, creating this execution flow:

```
XYCConcentrate
     │
     └── runLoop() ──► ProtocolFeeOut
                           ├── inflates amountOut (100 → 111)
                           ├── runLoop() ──► Swap accounts 111 tokens
                           ├── restores amountOut (111 → 100)
                           └── transfers 11 tokens as fee
     │
```

```
    └── _updateScales() sees amountOut = 100, not 111
│
└── _transferOut() tranfers 100 tokens - total tokens transfered = 100 + 11 = 111
```

The pool records that it sent 100 tokens whereas it actually sent 111. Each swap with protocol fees widens this gap. Over time, recorded liquidity diverges from actual reserves, causing the pool to promise swaps it cannot fulfill.

Consider placing protocol fee instructions **before** `XYCConcentrate` so that `_updateScales` executes while amounts still reflect actual transfers. Alternatively, consider capturing the transferred amounts before restoration.

Update: *Resolved in pull request #75 and pull request #82. The team stated:*

> We implemented a **stateless** `_xycConcentrateGrowLiquidity2D` instruction and defined the **correct instruction ordering** in the README, including:
>
> `protocol fees`
>
> `concentrate`
>
> `flat`
>
> This makes the intended execution flow explicit and ensures the instruction is used within the expected setup. The maker must place the `protocol fees` instruction first in the strategy program.

H-02 Dynamic Balance Tracking Does Not Account for Instantly Transferred Protocol Fees

The `_dynamicBalancesXD` instruction maintains a persistent balance mapping to track token amounts across multiple swaps for a given order. After executing nested instructions via `ctx.runLoop()`, it updates the stored balances by adding `swapAmountIn` to `tokenIn` balance and subtracting `swapAmountOut` from `tokenOut` balance. These balances are subsequently used in AMM curve calculations to determine swap prices.

When `_dynamicBalancesXD` wraps protocol fee instructions such as `_dynamicProtocolFeeAmountInXD`, the balance tracking becomes incorrect. The `_feeAmountIn` function temporarily adjusts `ctx.swap.amountIn` for the nested swap calculation, but then restores it to the original value (`ExactIn`) or increases it by the fee amount (`ExactOut`). The fee is then transferred directly from the maker to the fee recipient.

When control returns to `_dynamicBalancesXD`, the returned `swapAmountIn` reflects this restored/increased value, which gets recorded in the balance mapping.

However, the maker no longer possesses the fee portion as it was already transferred to the fee recipient. For example, if a user swaps with `amountIn = 1000` tokens and a 1% protocol fee applies, `10` tokens are transferred to the fee recipient while `990` tokens go to the pool. The balance mapping incorrectly records `+1000` instead of `+990`, overstating the pool's actual balance by the fee amount. This error accumulates with each fee-bearing swap, leading to progressively incorrect price calculations.

Consider either returning the fee amounts from protocol fee instructions so `_dynamicBalancesXD` can adjust the balance updates accordingly, or documenting that `_dynamicBalancesXD` should not be used in combination with instantly-transferring protocol fee instructions.

Update: *Resolved in pull request #75. The team stated:*

> We defined a **strict instruction ordering** in the README, requiring the `protocol fees` instructions to be placed **first** in the maker's strategy program.
>
> With this ordering, all subsequent instructions receive the **correct amount**, which ensures that the strategy accounting remains correct.
>
> In addition, we introduced an extra register, `amountNetPulled`, in the `SwapRegisters` struct of `SwapVM`, which allows the VM to correctly account for the amount of protocol fees pulled out during execution.

Medium Severity

M-01 Strategy Reinitialization via `ship` Allows Breaking Token Limits and Docking Invariants

The `ship` function has been designed to initialize a strategy once, but the current implementation allows the function to be executed multiple times for the same strategy hash as long as the token sets are disjoint. The validation logic only checks if the individual token balance has a token count of zero, failing to verify if the strategy hash itself has already been registered. This oversight allows a maker to append new tokens to an existing strategy by calling ship repeatedly with different token arrays. Consequently, a user can bypass the

immutability requirement and circumvent the maximum limit of 254 tokens by adding them in separate batches, as the length check only applies to the array in the current transaction.

This flaw also undermines the consistency of the docking mechanism. The protocol assumes that a strategy is either fully active or fully docked. However, if a strategy is constructed via multiple ship calls with different tokens, the maker can subsequently only dock a subset of those tokens. This leaves the strategy in a mixed state where some tokens are marked as docked while others remain active and tradable. This violates the invariant that docking a strategy should universally deactivate it.

Consider implementing a state variable or mapping to track whether a strategy hash has already been initialized for a given maker and app. By checking this flag at the beginning of the ship function, the protocol can ensure that a strategy hash is used exactly once.

Update: *Acknowledged, not resolved. The team stated:*

> The contract enforces immutability on a per-token basis: once a token is shipped for a given `(maker, app, strategyHash)`, any attempt to ship the same token again reverts (`balance.tokensCount != 0`). Therefore, already shipped tokens cannot be modified or overwritten.
>
> It is true that `ship()` can be called multiple times with the same `strategyHash` as long as the new token set is fully disjoint. This behavior is acceptable by design because the protocol does not assume `strategyHash` uniquely represents a single global token set unless the app builder encodes it that way.
>
> We recommend Aqua App Builders to include the set of tokens used by the strategy into the `strategy` calldata itself. In that case, `strategyHash = keccak256(strategy)` becomes unique for each strategy-token-set combination, eliminating any ambiguity.
>
> Even without this recommendation, takers remain protected by design: a taker acts based on profit incentive and can choose to interact only with the intended token pairs. We also recommend providing `quote()` / simulation functionality so the taker can validate the exchange flow and expected outcome before execution.
>
> Additionally, introducing extra storage slots (e.g., an `initialized` flag per `(maker, app, strategyHash)`) would increase gas costs for `ship()` and related flows. Higher transaction costs directly reduce the economic efficiency of the protocol and decrease the net profitability for participants. For this reason, we intentionally avoid additional persistent state unless it provides a clear security benefit.

M-02 Unrestricted `Aqua.push` Allows Direct Balance Manipulation

`Aqua.push` updates a maker's balance and transfers tokens as part of the swap flow. Unlike `Aqua.pull`, it is intentionally callable by external accounts to support callback-based integrations. Since `Aqua.push` is externally callable and accepts a `maker` parameter, a maker (or any account they control) can invoke it directly with their own address. This updates internal balances and triggers transfers without a validated app context. A check like `msg.sender != maker` would be insufficient, as a different account controlled by the maker could still perform the call. Moreover, a self-transfer results in no net balance change for the maker, allowing them to manipulate the flow without actual fund movement.

Consider allowing apps to flag when `push` is permitted, ensuring that balances can only be modified through the respective app:

```
AQUA.togglePush(maker, strategy, token, true);
msg.sender.callback(); // or AQUA.push(...);
AQUA.togglePush(maker, strategy, token, false);
```

This provides flexibility for apps to either allow donations by implementing a dedicated function or restrict balance changes to specific actions (e.g., swap, stake).

Update: *Acknowledged, not resolved. The team stated:*

> `Aqua.push` is intentionally externally callable to support callback-based integrations, and this behavior is by design.
>
> In the scenario described, if the maker (or an address controlled by the maker) calls `Aqua.push` for the maker's own strategy, the effect is maker-side only. Adding funds outside the intended curve/invariant does not create an exploit against takers; instead, it can create an arbitrage opportunity that is economically unfavorable to the maker (i.e. a self-inflicted loss).
>
> If a third party calls `Aqua.push` and transfers tokens into the maker's strategy, this is effectively a donation to the maker. It does not create direct harm to the maker or takers and does not allow extraction of funds from other participants.
>
> For these reasons, we do not consider this a vulnerability in the protocol. The ability to call `push` externally is a deliberate tradeoff to support integrations with callbacks.

M-03 Missing Validation for `begin < end` in `Calldata.slice` Allows Underflow

The `Calldata.slice()` function is used throughout the protocol to extract data slices based on user-controlled index values, notably in `MakerTraitsLib._getDataSlice()` and `TakerTraitsLib._getDataSlice()`. These slices are converted into `CalldataPtr` pointers via `CalldataPtrLib.from()` for use by the SwapVM during execution.

The function validates that `end <= calls.length` but does not verify that `begin <= end`. When `begin > end`, the operation `sub(end, begin)` underflows, producing a length close to `2^256`. When this corrupted slice is passed to `CalldataPtrLib.from()`, which packs offset and length into a `uint256` via `or(shl(128, data.offset), data.length)`, the underflowed length exceeds `uint128.max` and overflows into the upper 128 bits, corrupting the offset. When `runLoop()` uses this pointer, the corrupted offset and unbounded length cause the VM to read arbitrary calldata as opcodes, potentially resulting in denial of service or manipulation of swap computations.

Consider adding validation to ensure that `begin <= end` before computing the slice length.

Update: *Acknowledged, not resolved. The team stated:*

> We intentionally keep this library as gas-cheap as possible, so we do not plan to add an additional `begin <= end` check at this level. In our design, strategies must be validated before use, and takers have to interact only with validated strategies.
>
> This validation is responsible for rejecting malformed slices (including invalid begin/end ranges). In addition, we recommend using SwapVM.quote() to verify the expected swap result before execution. This provides an extra safety layer for integrators and helps detect malformed or unsafe strategy data before submitting the swap.

M-04 Aqua Protocol Fee Opcodes Cause Double Fee Payment in Direct Push Mode

When using Aqua-based orders, takers can settle swaps by pushing tokens directly to the maker's Aqua balance. The contract verifies this by comparing the current balance against `originalAquaBalanceIn`, a snapshot captured before `runLoop()` executes. The `_aquaProtocolFeeAmountInXD` opcode calls `AQUA.pull()` during `runLoop()` to transfer fees from the maker's Aqua balance. This reduces the balance below the snapshot value. The subsequent balance check (`balanceIn >= originalAquaBalanceIn +`

`amountIn`) then requires the taker to push extra tokens to compensate for the pull, effectively paying the fee twice.

Example (1:1 swap, 10% fee, taker wants 90 tokenOut):

- **Maker's Aqua balance**: 1000
- **`originalAquaBalanceIn` captured**: 1000
- **Computed `amountIn`**: 100 (90 base + 10 fee)
- Fee opcode pulls 10 → balance becomes 990
- **Check requires:** `990 + takerPush >= 1000 + 100`
- **Taker must push**: 110 (instead of expected 100)

Consider capturing `originalAquaBalanceIn` after `runLoop()` completes, or adjusting the balance check to account for Aqua balance changes during program execution.

Update: *Resolved in pull request #75. The team stated:*

> We defined a **strict instruction ordering** in the README, requiring the `protocol fees` instructions to be placed **first** in the maker's strategy program. With this ordering, all subsequent instructions receive the **correct amount**, which ensures that the strategy accounting remains correct.
>
> In addition, we introduced an extra register, `amountNetPulled`, in the `SwapRegisters` struct of `SwapVM`, which allows the VM to correctly account for the amount of protocol fees pulled out during execution. In this case also we changed this requirement: `require(balanceIn >= originalAquaBalanceIn + ctx.swap.amountIn - ctx.swap.amountNetPulled, AquaBalanceInsufficientAfterTakerPush(...));`

M-05 Extruction Can Violate the Quote/Swap Consistency Invariant

The `Extruction` instruction delegates pricing/state logic to a maker-chosen external `target`. During `quote()`, the VM executes `IStaticExtruction(target).extruction()` (a `view` path), while during `swap()` it executes `IExtruction(target).extruction()` (a non-`view` path), and these two interfaces may have different implementations or behavior. In addition, the `target` can be maliciously formed or updated later using an upgradeable contract such that its logic can change between a taker's `quote()` and subsequent `swap()`. As a result, the protocol cannot enforce quote/swap amount equivalence for Extruction-based strategies.

Takers may select orders based on a favorable `quote()` that later executes with different amounts or reverts in `swap()`, leading to degraded UX, misrouting, and denial-of-service style failures. While taker-side slippage limits can prevent unexpected fills, this still breaks the *Quote/Swap Consistency* invariant and can cause unexpected execution outcomes.

Consider adding protocol-level safeguards to omit quote/swap divergence for `Extruction` (e.g., ensuring the `target` logic cannot change between `quote()` and `swap()`), or clearly documenting that programs with `Extruction` instructions may cause different behavior and amounts between `quote()` and `swap()`.

Update: *Resolved. The team stated:*

> We agree that Extruction introduces flexibility that can lead to quote/swap inconsistencies if makers implement non-deterministic or upgradeable target contracts. This is an intentional design decision to enable advanced custom strategies.
>
> Mitigation: - Adding comprehensive NatSpec documentation to warn takers/resolvers about validation requirements - Documenting that target contracts should be immutable and deterministic - Takers already have slippage protection mechanisms that mitigate exploitation - This is a "use at your own risk" feature - takers should only interact with validated strategies
>
> Takers/resolvers are expected to: 1. Verify target contract code before interaction 2. Ensure target is non-upgradeable or has trusted governance 3. Validate quote/swap consistency in their risk assessment 4. Apply appropriate slippage tolerances

M-06 Signing Loose Non-Aqua Strategy Affects All Non-Aqua Strategies of the Maker

The `orderHash` generated in the SwapVM does not utilize `tokenIn` and `tokenOut` addresses given as input by `taker`. While the Aqua protocol requires the strategies to be shipped with the token, the non-Aqua strategy (i.e., the strategy verified using a signature) does not have any constraints on which tokens will be utilized.

Assume a maker that has multiple non-Aqua strategies signed and active in the SwapVM and the underlying tokens are already approved to the SwapVM. If the maker adds a loose strategy (e.g., signing a order without the `balances` instruction), the taker can target all the approved tokens to the SwapVM by changing the `tokenIn` and `tokenOut` addresses while swapping using the loose strategy. These tokens could have been **only approved for a particular strategy** on the `SwapVM`. In this scenario, all non-Aqua strategies are dependent on each

other and are highly error-prone to new types of instruction versions and new strategies added by the maker.

This PoC, showcases how tokens of perfectly fine `strategy1` and `strategy2` are swapped using a loose `strategy3`, bypassing the ideal behavior of `strategy1` and `strategy2`.

Consider adding protocol-level protection for non-Aqua strategies ensuring that all relevant tokens are considered when generating the `orderHash`. Alternatively, consider thoroughly documenting this interdependency of non-Aqua strategies.

Update: *Resolved in pull request #85. The team stated:*

> Added comprehensive "Strategy Hash Uniqueness and Token Safety" section to README with:
>
> - Clear warning about mandatory balance instructions for non-Aqua strategies
> - Maker have to create strategies with unique orderHash - using _salt() instruction
> - When maker uses custom accounting model in _extruction() he have to ensure that all relevant tokens are considered when generating the `orderHash`
> - Safe vs unsafe code examples
> - Risk scenarios with mitigations
> - Best practices summary

M-07 Decay Stores Pre-Fee `amountIn` Creating Asymmetric Price Impact Protection

In `AquaAMM`, instructions execute in the following sequence: `XYCConcentrate` → `Decay` → `Fee` → `XYCSwap`. The `Decay` instruction provides temporary price impact protection by storing trade amounts as offsets after calling `runLoop()`, which are later applied to balances on reverse-direction trades to neutralize arbitrage opportunities. The `Fee` instruction temporarily reduces `amountIn` for the swap calculation, then restores it to the original value afterward.

Since Fee restores `amountIn` before control returns to Decay, the stored offset for `tokenIn` reflects the pre-fee amount instead of the post-fee amount actually used in the swap. For example, if a trader swaps 500 tokens with a 1% fee, `XYCSwap` computes `amountOut` based on 495 tokens, but Decay stores 500 as the offset. On a subsequent reverse trade, Decay subtracts 500 from `balanceOut` instead of 495, over-compensating by the fee amount. This discrepancy is further amplified when using `_aquaProtocolFeeAmountInXD`, which immediately transfers the fee to the recipient. In this case, the maker's actual balance

increases by `amountIn - fee`, yet Decay stores the full `amountIn` as the offset, creating an even larger mismatch between the recorded price impact and the actual balance change.

Consider storing the decay offsets based on the post-fee `amountIn` value. This could be achieved by having Decay capture `amountIn` after Fee has processed it, or by adjusting the instruction order so that Decay's offset storage occurs before Fee restores the original value.

Update: *Resolved in pull request #75. The team stated:*

> We described correct order of instructions in Readme. Also added `test/SwapVmAccounting.t.sol and test/AquaAccounting.t.sol` for comprehensive conservation checks validating this behavior.

M-08 Hook Token Injection Bypasses Concentrated Liquidity Price Bounds

The `XYCConcentrate` instruction implements concentrated liquidity by adding virtual "delta" balances to real token reserves. These deltas are computed via `computeDeltas` using the formula `deltaA = balanceA / (√(price/priceMin) - 1)`, which inflates the effective pool size used in the constant product (x*y=k) swap calculation. This reduces price impact for traders while concentrating the maker's capital within a defined price range `[priceMin, priceMax]`. The `concentratedBalance` function returns `balance + delta` on the first swap, or scales the delta proportionally to current liquidity on subsequent swaps. The delta values are mathematically designed such that when the price reaches a bound, the corresponding real token balance depletes to zero. At this point, any further swap attempting to extract that token would revert due to insufficient balance during the ERC20 transfer at `_transferOrPull`, naturally enforcing the price range without explicit validation.

The SwapVM executes maker hooks and taker callbacks via `_transferIn` and `_transferOut` between the program computation and the actual token transfers. Specifically, the maker's `preTransferOutHook` and the taker's `preTransferOutCallback` execute after the `runLoop` completes swap amount computation but before the outbound `safeTransferFrom` occurs. Either party can configure a hook contract implementing `IMakerHooks.preTransferOut` or `ITakerCallbacks.preTransferOutCallback` that injects the output token into the maker's balance during this window. Since the swap amount was calculated by `_xycConcentrateGrowLiquidity2D` using virtual balances (which showed sufficient liquidity via the delta), and the hook provides real tokens for the transfer, the swap succeeds despite the pool being at its price bound. This pushes the effective price below `priceMin` (or

above `priceMax`), violating the concentrated liquidity invariant. The `_updateScales` function then persists an inconsistent liquidity state to the `liquidity` mapping, as it updates based on computed concentrated amounts rather than actual post-hook real balances.

Numerical Example

Consider a pool with `priceMin = 0.25` configured via `deltaA = 1000` and `deltaB = 1000`. After several swaps, the pool reaches its lower bound:

State	Real BalanceA	Real BalanceB	Concentrated A	Concentrated B	Effective Price
At `priceMin`	3,000	0	4,000	1,000 (virtual)	0.25

An attacker initiates a swap to buy 100 tokenB. The VM computes `amountIn = (100 × 4000) / (1000 - 100) = 444 tokenA` using virtual balances. Normally, the transfer would revert since `real_balanceB = 0`. However, the attacker's `preTransferOutCallback` injects 100 tokenB into the maker's balance before the transfer executes. The swap succeeds, resulting in:

State	Real BalanceA	Real BalanceB	Concentrated A	Concentrated B	Effective Price
Post-attack	3,444	0	4,444	1000 (virtual, does not change)	**~0.22 < priceMin**

Consider validating that real token balances are sufficient to cover the computed `amountOut` before executing the transfer, independent of concentrated balance calculations. Alternatively, recalculating and verifying price bounds after hooks execute would ensure the invariant `priceMin ≤ effectivePrice ≤ priceMax` is maintained. Another approach would be restricting hook capabilities when concentrated liquidity instructions are active, preventing external balance manipulation during the critical transfer phase.

Update: *Acknowledged, will resolve. The team will add documentation notifying makers to not interact with Aqua through hooks.*

M-09 Protocol Fee on `amountIn` Is Pulled From the Maker Before the Taker's `tokenIn` Transfer, Risking Swap Reverts

The `fee-on-amountIn` opcodes (E.g., `_protocolFeeAmountInXD`, `_aquaProtocolFeeAmountInXD`, `_dynamicProtocolFeeAmountInXD`, and `_aquaDynamicProtocolFeeAmountInXD`) perform the transfer of fees from the `maker` inside `ctx.runLoop()` (i.e., before `SwapVM.swap()` executes the taker→maker `tokenIn` transfer). If the `maker` does not already have sufficient `tokenIn` balance/allowance, the swap reverts even though the maker would have received `tokenIn` later in the same transaction. This can cause *unexpected swap reverts* and also **quote/swap** divergence in practice for strategies that apply protocol fee on `amountIn`, reducing liveness/UX.

Consider redesigning fee collection so it is taken after the taker's `tokenIn` transfer to avoid reliance on the maker's pre-existing `tokenIn` balance or document this requirement explicitly (protocol-fee-on `amountIn` requires the maker to pre-fund/approve `tokenIn`).

Update: *Acknowledged, not resolved. The team stated:*

> We are aware of this behavior and consider it an accepted design constraint for this strategy type.
>
> For AMM strategies, this is acceptable: the maker creates a strategy for a token pair, and the strategy itself does not lock assets (the maker may dock() at any time). In this model, having some tokenIn balance/allowance available on the maker side is a natural operational requirement of running a two-sided pool.
>
> In other words, protocol-fee-on-amountIn is expected to operate with maker pre-funding / pre-approval of tokenIn, and swaps may revert if the maker does not maintain sufficient

M-10 Multiple Fee Instructions Compound Causing `Fee-on-Fee` for Swaps During `isExactOut` Path

In `_feeAmountIn` (`ExactOut` branch), the fee instruction first executes the remainder of the VM program via `ctx.runLoop()`, then computes `feeAmountIn` from the computed `ctx.swap.amountIn`, and finally increases `ctx.swap.amountIn` by `feeAmountIn`. Consequently, if multiple `fee-on-amountIn` instructions are included in the same program,

the outer fee is computed on an `amountIn` that may already include inner fee adjustments (`fee-on-fee`), making the total fee *order-dependent* and effectively *compounding* rather than being calculated once on the raw swap amount.

This can lead to multiple scenarios as follows:

- **Unexpected Economics:** total fees higher than an additive interpretation, including `fee-on-fee`,
- **Configuration Order:** reordering fee instructions changes the effective price/fees, amplified by rounding,
- **Integration/UX Risk:** off-chain quoting and strategy configuration may assume fees apply to the raw swap amount only and thus mis-estimate execution amounts or fee splits when multiple fee instructions are present.

Consider redesigning the fee collection logic to accumulate fee parameters in context and settle once based on the raw swap-computed `amountIn` (pre-fee), ensuring fee calculation is independent of instruction nesting/order.

Update: *Acknowledged, not resolved. The team stated:*

> This behavior is **intentional and fundamental** to SwapVM's composable architecture. It provides makers with maximum flexibility to construct sophisticated fee structures while maintaining execution model consistency.
>
> Both makers and takers must understand that instruction composition creates fee-on-fee effects, and actual swap amounts should always be verified via `quote()` before execution.

M-11 `AquaAMM` Forwards `uint16` Fees Into `BPS = 1e9` Fee Math, Capping Fees Below 1 BPS

The `AquaAMM.buildProgram` helper builds a `SwapVM` program and appends fee instructions by encoding arguments with `FeeArgsBuilder.buildFlatFee` and `FeeArgsBuilder.buildProtocolFee`. Fee opcodes interpret `feeBps` under the non-standard denominator `BPS = 1e9` (as also documented in the args format comments for `_flatFeeAmountInXD`).

Since `feeBpsIn` and `protocolFeeBpsIn` are `uint16` values, the maximum representable fee is `65535/1e9` (0.0065535%), which is below 1 conventional basis point (1 bps would require `feeBps = 1e5`). This makes typical fee tiers (for example, 1-100 bps) impossible to

express through `AquaAMM.buildProgram`. Furthermore, supplying values under the conventional `1e4` basis-point convention silently undercharges maker and protocol fees by 100,000x, causing systematic fee under-collection and mispricing. The `protocolFeeBpsIn <= feeBpsIn` check does not mitigate this unit mismatch.

Consider changing `feeBpsIn` and `protocolFeeBpsIn` to `uint32` (or `uint256`) and documenting that the expected scale is `BPS = 1e9`. Consider also renaming inputs to reflect the `1e9` scale and validating that the chosen representation can express intended fee tiers.

Update: *Resolved. The `AquaAMM.buildProgram` helper was removed from SwapVM repository.*

M-12 `quote` Is Not a Faithful Simulation of `swap` due to `isStaticContext`-Dependent Opcode Semantics

The documentation defines Quote/Swap Consistency as a core invariant and describes `quote()` as a view preview whose returned amounts must match `swap()`. In the implementation, `SwapVM.quote()` explicitly constructs the VM context with `isStaticContext: true`, while `SwapVM.swap()` sets `isStaticContext: false`. The program execution engine, `ContextLib.runLoop()`, executes arbitrary opcodes and trusts `ctx.vm.nextPC` updates made by those opcodes.

Multiple instructions branch on `ctx.vm.isStaticContext`, making quote-mode and swap-mode semantics materially different. For example, fee instructions condition real token movement on `!isStaticContext` (e.g., `Fee._protocolFeeAmountInXD`), so `quote()` can succeed even when `swap()` later reverts due to missing approvals or balances. Stateful instructions similarly skip persistence in quote-mode (e.g., `Invalidators._invalidateBit1D`), enabling control-flow differences when combined with backwards jumps (see `Controls._jump`) and the run loop program-counter handling.

Example

- A program executes `_invalidateBit1D`, then uses `_jump` to execute `_invalidateBit1D` again.
- `quote()` succeeds because `_invalidateBit1D` does not persist the bit in static context.
- `swap()` reverts on the second `_invalidateBit1D` because the bit is persisted after the first call.

Consider documenting all such instructions and scenarios that may break the `Quote/Swap` consistency to ensure that takers are aware of them.

Update: *Resolved in pull request #86. Documentation was updated for such cases.*

Low Severity

L-01 Emission of Misleading Events When Processing Empty Arrays in `ship` and `dock` Functions

The `ship` and `dock` functions execute without verifying that the provided token arrays contain at least one element. If a caller invokes these functions with an empty array, the contract emits `Shipped` or `Docked` events but skips the logic inside the loops that handles balance updates and storage modifications. As a result, the blockchain records an event signaling that a strategy was deployed or deactivated, while strictly speaking, no operations were performed on any liquidity positions or balances.

This discrepancy introduces noise into the event log and can mislead off-chain monitoring tools that rely on these signals to track the protocol state. Indexers and user interfaces may register and display these strategies as valid actions, creating confusion or data inconsistency where the event history implies activity that is not reflected in the contract storage.

Consider adding a `require` statement at the start of both functions to validate that the input array length is greater than zero.

Update: *Acknowledged, not resolved. The team stated:*

> We intentionally do not enforce an explicit `tokens.length > 0` check in `ship()` / `dock()` to avoid additional gas overhead for all honest users. A rational and honest maker has no incentive to initialize an empty strategy, as it provides no liquidity positions, no balances, and no execution benefit.
>
> A malicious or spammy maker may still emit such events, but they pay the full transaction cost themselves. This does not impact protocol safety or on-chain state integrity, since no storage modifications occur when the token array is empty.

> Indexers, integrators, and aggregators are expected to validate strategies against on-chain storage and ignore "empty" event-only strategies (i.e., cases where no token balances exist under `(maker, app, strategyHash)`). This approach keeps the core protocol cheaper for honest participants while allowing off-chain consumers to filter out noise deterministically.

L-02 `dock` Accepts `tokens` Array of Length `_DOCKED`, Enabling Continuous Event Emission

When a maker docks a strategy via `dock`, the strategy's balance is reset to 0 and `tokensCount` is set to the sentinel value `_DOCKED`. Off-chain indexers typically rely on emitted events to reflect state transitions, such as docking, and may parse the `tokens` array provided by the caller to understand which assets are involved.

The current implementation does not prevent callers from supplying a `tokens` array whose length equals `_DOCKED`. A caller can populate this array (even with duplicate entries), pass existing validations, and still trigger the event, despite the internal state being "docked" with `tokensCount` set to `_DOCKED`. This leads to events that suggest asset details that do not correspond to the docked state, potentially misleading off-chain consumers.

Consider reverting when `tokens.length == _DOCKED` to prevent emitting events with misleading `tokens` data during docking. Remember not to remove the `balance.tokensCount == tokens.length` check, as that could still allow a bypass.

Update: *Acknowledged, not resolved. The team stated:*

> In our implementation, `Docked` events do not include token details (only `(maker, app, strategyHash)`), therefore off-chain systems must not treat the calldata `tokens[]` as a canonical source of truth for the docked asset set.
>
> Additionally, `tokens.length == _DOCKED` cannot be used to dock an active strategy because `ship()` explicitly prevents initializing a strategy with `_DOCKED` token count. The only scenario where `tokens.length == _DOCKED` passes validation is when the provided tokens are already in a docked state (`balance.tokensCount == _DOCKED`), which results in a no-op state transition and does not impact protocol correctness.
>
> Adding extra checks to revert on `tokens.length == _DOCKED` would introduce additional gas overhead for honest participants without providing a meaningful security benefit. Integrators and indexers are expected to validate strategy/token state against

> on-chain storage and ignore non-canonical calldata representations (e.g., duplicated tokens).

L-03 Missing Validation for Incompatible Aqua Order Configurations in `MakerTraitsLib.build()`

The `MakerTraitsLib.build` function constructs swap orders by encoding various configuration flags into the `MakerTraits` type. One such flag, `useAquaInsteadOfSignature`, enables Aqua-based order authentication where balances are managed through the Aqua protocol instead of requiring a maker signature. However, this mode has specific constraints: it cannot be combined with the `shouldUnwrapWeth` flag, and the order receiver must be the maker address.

The `build()` function does not validate these incompatibilities when constructing the order. While the `SwapVM.swap()` function does enforce these constraints at execution time, reverting with `MakerTraitsUnwrapIsIncompatibleWithAqua` or `MakerTraitsCustomReceiverIsIncompatibleWithAqua` errors, this late validation means users can construct invalid orders that will inevitably fail upon swap execution.

Consider adding validation checks in `MakerTraitsLib.build()` to revert early when `useAquaInsteadOfSignature` is set to `true` alongside `shouldUnwrapWeth` or a custom receiver address. This would provide clearer error feedback at order construction time rather than at swap execution.

Update: *Acknowledged, not resolved.*

> *The team clarified that the referenced logic is test-only (used in Foundry), removed from deployed bytecode as dead code, never invoked on-chain in production, and not used by SDK/provider order creation flows (which are off-chain). They consider parameter compatibility a maker/SDK responsibility, with protocol guarantees enforced at execution time.*

L-04 Debug Opcode Included in Production Bytecode Increases Gas Costs

The `buildProgram` function in `AquaAMM` constructs bytecode that defines the swap logic for orders. This bytecode is built by concatenating various instructions such as fee

calculations, decay handling, and swap execution. The function includes a `_printContext` debug opcode in the production bytecode. This debug instruction, which logs internal VM state using `console.log`, serves no functional purpose in a production environment and will consume unnecessary gas each time an order is executed.

Consider removing the `program.build(_printContext)` call from the bytecode construction to reduce gas costs.

Update: *Resolved in pull request #74.*

L-05 Balance Underflow in Dynamic Balances

In the `_dynamicBalancesXD` instruction, the `swapAmountOut` is deducted from `balances` based on the `orderHash` and `tokenOut`. However, if the `swapAmountOut` is greater than `balances` stored, the swap would fail. As this deduction only occurs when the context is not `static` (i.e., `isStaticContext` variable is set to `false`), the `quote` function would return the value while the `swap` function would revert without any errors, thus breaking the invariant in the documentation stating that `quote()` and `swap()` should return identical amounts.

Consider checking against the `balances` first before deduction and revert with a meaningful error to show consistent behavior between the `quote` and `swap` functions.

Update: *Acknowledged, not resolved. The team stated:*

> We intentionally omit this check `require(swapAmountOut <= balances[...][tokenOut])` to minimize protocol gas costs. **The current design is optimal** for gas efficiency. Takers must exercise due diligence when specifying swap amounts, especially in ExactOut mode.

L-06 Asymmetric Decay Rounding Creates Unfavorable Trading Rates for Low-Decimal Token Pairs

The `Decay` instruction provides temporary price impact protection by storing trade amounts as offsets that linearly decay over time. The decay calculation uses the formula `offset * timeLeft / decayPeriod`, and on reverse trades, these offsets are applied to balances where `balanceIn` is increased and `balanceOut` is decreased to simulate reduced price impact from the previous trade.

When a pool contains tokens with significantly different decimal precisions (e.g., BTC with 8 decimals paired against ETH with 18 decimals), the low-decimal token's offset rounds to zero before the high-decimal token's offset expires. For example, with a trade producing a BTC offset of 3,333 satoshis and an ETH offset of 1×10^{15} wei, and a decay period of 65,535 seconds, the BTC offset becomes zero when `timeLeft < 19 seconds` while the ETH offset still holds 274,662,394,140 wei.

During this asymmetric window, a reverse trade (BTC→ETH) has `balanceIn` unchanged (no boost from BTC offset) while `balanceOut` is still reduced (ETH offset active). This results in the trader receiving approximately 0.09% less output compared to trading after full decay expiry. While the impact per trade is negligible and the window is brief (19 seconds out of ~18 hours), the asymmetry creates a subtle bias against traders who happen to execute during this period.

Consider scaling decay offsets by a constant precision factor before storage and applying inverse scaling when reading. Doing so will ensure that both tokens decay at equivalent rates regardless of their native decimal precision.

Update: *Acknowledged, not resolved. The team stated:*

> This is an expected consequence of integer math and heterogeneous token decimals: smaller-precision offsets will quantize to zero slightly earlier than higher-precision offsets near the end of the decay window.The Decay instruction is designed as a lightweight, best-effort mechanism for temporary price-impact protection, not as a precision-perfect symmetrical decay model across arbitrary decimal regimes. Given the negligible magnitude and brief duration of the asymmetry, we do not plan to change the current implementation.

L-07 Missing Zero-Address Checks

When operations with `address` parameters are performed, it is crucial to ensure that the provided `address` value is not zero. Setting an `address` variable to zero is problematic because it has special burn/renounce semantics or can cause miscellaneous reverts.

Throughout the codebase, multiple instances of missing zero-address checks were identified:

- The `aqua` operation within the `SwapVM` contract
- The `tokenIn` and `tokenOut` parameters during the `AQUA.safeBalances` operation in the `quote` function of the `SwapVM` contract

- The `tokenIn` and `tokenOut` parameters during the `AQUA.safeBalances` operation in the `swap` function of the `SwapVM` contract
- The `aqua` operation within the `Fee` contract

Consider always performing a zero-address check before assigning a state variable.

Update: *Acknowledged, not resolved.*

L-08 Setting `feeBPS == BPS` Causes Swap to Revert And Missing Runtime Validation for `feeBPS`

The `build` function of the `Fee` instruction currently performs checks which ensure that `feeBPS <= BPS`. This allows for fees of up to **100%** of the swap amount.

However, setting `feeBPS == BPS` leads to problematic behavior:

- In the `isExactOut` path, this results in a division-by-zero, causing the swap to revert without an explicit error.
- In the `isExactIn` path, the computed `amountIn` becomes zero, which can cause downstream instructions (e.g., `XYCConcentrate`) to fail, as they do not allow a zero `amountIn`.

Moreover, the `parse*` functions do not validate whether the decoded `feeBps` is actually less than `BPS`, leading to EVM reverts without errors.

To prevent the above-listed edge cases, consider updating the check in the `Fee:build` function to enforce that `feeBPS < BPS` and adding runtime validations for the fees.

Update: *Acknowledged, not resolved.*

L-09 Allow Makers to Revoke a Signature to Protect Other Non-Aqua Strategies

Once a maker creates a signature for a non-Aqua strategy, the signature cannot be revoked. A loose strategy could have a ripple effect on other strategies, forcing the maker to completely get out of the `SwapVM` system to safeguard their funds. The `Invalidators` instructions only provide support to the maker if they are included in the program. If the program does not

include `Invalidator` instructions, there is no way for the maker to redo the leaked signature.

Consider implementing a functionality that allows makers to disqualify a signature.

Update: *Acknowledged, will resolve.*

L-10 `unwrapWeth` Does Not Require `Token` to Be WETH

`SwapVM` supports optional WETH unwrapping during settlement. When `takerTraits.shouldUnwrapWeth()` is set, `_transferFrom` first transfers the ERC-20 `token` into `address(this)` and then calls `IWETH(token).safeWithdrawTo(amount, to)`. The `token` value comes from the swap parameters (e.g., `tokenOut` in `swap`).

No check enforces that `token` equals the WETH address configured in the `SwapVM` constructor. As a result, any contract can be used as `tokenOut` while still hitting the unwrap branch. If `token.withdraw(amount)` does not transfer ETH into `SwapVM`, the `safeWithdrawTo` call can still send `amount` ETH from the existing `SwapVM` ETH balance to `to`.

More than 5100 ERC-20 tokens have been identified on the Ethereum Mainnet where performing the `token.withdraw(uint256 amount)` function call is feasible. These tokens are generally vault tokens with an underlying asset, calling `withdraw` on these tokens could lead to the underlying asset getting stuck in `SwapVM`. Moreover, this allows `maker` to create malicious tokens to transfer ETH from `SwapVM` to themselves, but since `SwapVM` is not supposed to hold ETH this issue is identified as low.

Consider enforcing `token == WETH` before entering the unwrap branch (e.g., by storing the canonical WETH address in `SwapVM`).

Update: *Acknowledged, will resolve.*

L-11 `AquaAMM` Imports `ProgramBuilder` from `test/`, Breaking Builds That Only Ship `src/`

The `AquaAMM` strategy builder is a user-facing helper that constructs `SwapVM` programs and returns an `ISwapVM.Order`. It is included under `src/` and is documented for direct consumption in `README.md`. However, `AquaAMM` imports `ProgramBuilder` from `test/`

using a relative path. Relative imports bypass remappings, and common publishing setups intentionally exclude `test/` sources. This makes compilation and deployment packaging brittle and can introduce an unintended dependency on test-only helpers.

Consider moving `ProgramBuilder` (or a minimal subset) under `src/` and updating `AquaAMM` to import it from that stable location. In addition, consider ensuring that the published artifacts include every imported source and do not require `test/` to compile.

Update: *Resolved in pull request #74.*

L-12 PeggedSwap Exact-Out Rounding Can Compute `amountIn == 0` for Non-Zero `amountOut`

The `_peggedSwapGrowPriceRange2D` function computes swap amounts using normalized reserves and an analytical solve. In the exact-out branch, it computes `y1 = y0 - amountOut * rateOut`, then `v1 = y1 * ONE / config.y0` (rounded down), then `u1 = PeggedSwapMath.solve(v1, ...)`, then `x1 = ceilDiv(u1 * config.x0, ONE)`, and finally `amountIn = ceilDiv(x1 - x0, rateIn)`.

Due to integer quantization, small changes in `amountOut` can fall into rounding plateaus where `v1` does not change (or changes too little to affect the integer output of `PeggedSwapMath.solve`). In these cases, `u1` and `x1` can remain unchanged, making `x1 - x0 == 0`, and therefore, `ctx.swap.amountIn == 0` for a non-zero `ctx.swap.amountOut`. If the maker enables `allowZeroAmountIn`, `MakerTraitsLib.validate` permits `amountIn == 0`, and settlement skips the input transfer due to the `if (ctx.swap.amountIn > 0)` check in `SwapVM._transferIn` while still transferring the output in `SwapVM._transferOut`. If `allowZeroAmountIn` is not enabled, the same plateau can make exact-out swaps/quotes revert (either due to the checked subtraction `x1 - x0` when `x1 < x0`, or due to maker validation), creating an implicit minimum trade size and quote/fill inconsistencies for dust-sized requests.

Consider adding an explicit guard in the exact-out branch to ensure that `amountOut > 0` implies `amountIn > 0` (for example, revert when `x1 <= x0` or clamp `x1` to at least `x0 + 1` before computing `amountIn`). Furthermore, consider documenting a minimum viable `amountOut` for exact-out `PeggedSwap` usage and avoiding `allowZeroAmountIn` for orders that rely on `PeggedSwap` exact-out behavior.

Update: *Resolved in pull request #98. The team updated documentation for* `allowZeroAmountIn` *cases.*

L-13 `XYCConcentrate` Can Brick an Order by Dividing by Zero After the First Successful Fill

`XYCConcentrate` concentrates liquidity by scaling `ctx.swap.balanceIn` and `ctx.swap.balanceOut` using token-specific `delta` values and a liquidity ratio. The scaling is performed by `concentratedBalance`, which uses `liquidity[orderHash]` as the current liquidity and uses `initialLiquidity` supplied via calldata parsing in `parseXD` or `parse2D`.

If `initialLiquidity` is encoded as `0`, the first interaction takes the `currentLiquidity == 0` branch and succeeds, but a non-static call updates `liquidity[orderHash]` to a positive value in `_updateScales`. Subsequent calls for the same `orderHash` still carry `initialLiquidity = 0` and take the division branch (`delta * currentLiquidity / initialLiquidity`), reverting with division by zero. This permanently DoS further fills and can also break quoting for the affected order.

Consider validating that `initialLiquidity != 0` during arg parsing and reverting early with a dedicated error.

Update: *Resolved in pull request #82. The latest version of* `XYCConcentrate` *is stateless and doesn't use* `_updateScales` *function.*

L-14 `PeggedSwap` Missing Token Address Validation Allows Incorrect Rate Application

The `PeggedSwap` instruction is designed to facilitate swaps between pegged assets (such as stablecoins) using a square-root linear curve formula. The instruction arguments include rate multipliers (`rateLt` and `rateGt`) that normalize tokens with different decimals. These rates are assigned based on token address comparison during execution via the `getRates` function—the token with the lower address uses `rateLt`, and the token with the greater address uses `rateGt`.

The `build` function only encodes the curve parameters (`x0`, `y0`, `linearWidth`, `rateLt`, `rateGt`) without including the intended token addresses. Unlike

`XYCConcentrateArgsBuilder.buildXD`, which explicitly encodes token addresses in its arguments, `PeggedSwap` solely relies on address comparison at execution time.

If a maker holds balances in tokens other than the two intended for the pegged swap, a taker could potentially execute swaps against unintended token pairs. Since the rate multipliers were calibrated for specific token decimals (e.g., USDC with 6 decimals / DAI with 18 decimals requiring `rateLt = 1e12` and `rateGt = 1`), applying these rates to different tokens with mismatched decimals could result in incorrect pricing and fund drainage.

Consider encoding the intended token addresses within the instruction arguments and validating that `tokenIn` and `tokenOut` match the encoded addresses before applying the rate multipliers in `_peggedSwapGrowPriceRange2D`.

Update: *Acknowledged, not resolved. The team stated:*

> PeggedSwap is intentionally generic and does not encode token addresses. It must be used only in **pair-scoped** strategies.
>
> Non-Aqua: enforce the token set via `_staticBalances()` / `_dynamicBalances()`.
>
> Aqua: token set is enforced by the `ship()` token list (tokens outside the shipped set cannot be used).
>
> Also maker should pay attention for **order hash uniqueness** (i.e., the strategy program must uniquely describe the intended behavior / token universe).

L-15 `computeLiquidityFromAmounts` Reverts For Out-of-Range Spot Prices Despite Partial Handling

In `XYCConcentrate` instruction, the `computeLiquidityFromAmounts` function handles out-of-range spot prices in its own logic, setting `type(uint256).max` for the non-needed token, but then calls `computeBalances` which assumes in-range prices and reverts via unsigned underflow when `sqrtPspot` is outside `[sqrtPmin, sqrtPmax]`. This behaviour affects users computing position initialization parameters when spot price is outside the intended range.

Ideally consider reverting in `computeLiquidityFromAmounts` function if the condition `sqrtPmin <= sqrtPspot < sqrtPmax` is not satisfied during the computation,

alternatively consider adding boundary checks in `computeBalances` to return `bLt=0` when `sqrtPspot >= sqrtPmax` and `bGt=0` when `sqrtPspot <= sqrtPmin`.

Update: *Resolved in pull request #105.*

L-16 Fee Layer Violates Declared Price Bounds From Initial State

The fee instruction (Fee._flatFeeAmountInXD) is applied as a separate layer on top of the AMM invariant. The new stateless XYCConcentrate instruction correctly limits the AMM curve to the range [P_min, P_max], but the fee layer has no awareness of these declared bounds.

For `isExactIn=false` swaps, the fee is added after the AMM computes `amountIn`:

```
feeAmountIn = ctx.swap.amountIn * feeBps / (BPS - feeBps);
ctx.swap.amountIn += feeAmountIn;
```

Similarly, for `isExactIn=true`, the fee is already deducted before the swap calculation and then `amountIn is reset.

```
uint256 takerDefinedAmountIn = ctx.swap.amountIn;
feeAmountIn = ctx.swap.amountIn * feeBps / BPS;
ctx.swap.amountIn -= feeAmountIn;
ctx.runLoop();
ctx.swap.amountIn = takerDefinedAmountIn;
```

A maker declares the order will only execute in [2,000 – 4,000]. The actual execution range is [1,880 – 4,255] at 6% fee, or [1,940 – 4,127] at 3% fee. Takers pay prices outside the maker's intended bounds on every marginal trade near the extremes from the very initial state.

Consider documenting this scenario for the makers and takers to avoid confusion regarding actual price.

Update: *Resolved in pull request #105. The team stated:*

> It's a normal behavior of such types of AMM. Documentation added to highlight this behavior.

Notes & Additional Information

N-01 Inconsistent Modifier Documentation in `AquaApp`

The NatSpec documentation for the `AquaApp` contract provides incorrect instructions for using the `nonReentrantStrategy` modifier. While the comments suggest that the modifier accepts a single argument (`keccak256(abi.encode(strategy))`), the actual implementation requires two parameters: `address maker` and `bytes32 strategyHash`. This discrepancy can lead to integration errors and confusion for developers inheriting from this abstract contract.

Consider updating the NatSpec comments in the `AquaApp` contract to accurately reflect the required parameters for the `nonReentrantStrategy` modifier. The documentation should be aligned with the actual function signature and the underlying `_reentrancyLocks` mapping structure, which explicitly tracks locks per maker and strategy hash.

Update: *Resolved at PR #41.*

N-02 Incorrect Max Token Count Value When Exceeding Max Number of Tokens

The `ship` function ships strategies and sets initial token allowances for an app. The token list length is capped at `_DOCKED - 1` (254). When reverting with `MaxNumberOfTokensExceeded(uint256 tokensCount, uint256 maxTokensCount)`, the code passes `_DOCKED` as `maxTokensCount` instead of `_DOCKED - 1`, which is inconsistent with the enforced bound and can mislead integrators and monitoring tools.

Consider passing `_DOCKED - 1` as the `maxTokensCount` argument to `MaxNumberOfTokensExceeded` so the revert data reflects the actual limit, and add a unit test that asserts the emitted `maxTokensCount` equals `_DOCKED - 1`.

Update: *Resolved at PR #42.*

N-03 `safeBalances` Returns Duplicate Balances When Identical Tokens Are Provided

The `Aqua.safeBalances` function returns a strategy's balances for two supplied token addresses and checks that each token is initialized, and the strategy is not docked. Integrators may rely on it to obtain balances safely for accounting, quoting, and routing. However, the function accepts identical addresses for `token0` and `token1`. Hence, when the same token is passed twice, the call succeeds and returns the same balance in both outputs, which can cause callers to treat the two values as distinct and unintentionally double-count a single token.

Consider reverting when `token0 == token1` to enforce distinct tokens and prevent duplicated outputs.

Update: *Acknowledged, not resolved. The team stated:*

> `safeBalances()` is a low-level helper that returns balances for two user-supplied token addresses. Passing identical addresses is considered invalid input on the caller side. In such a case, the function will correctly return the same balance twice, which is expected behavior given both lookups reference the same storage slot.
>
> This does not introduce any on-chain inconsistency or protocol-level risk. Integrators and routers are expected to validate inputs (e.g., enforce `token0 != token1`) in their higher-level quoting and accounting logic.
>
> We intentionally avoid adding extra input checks to keep the function gas-efficient and because this is not a security issue but rather an integration guardrail.

N-04 Inconsistent Scaling Documentation in `PeggedSwapMath.solve`

The library performs fixed-point arithmetic using a shared scaling constant `ONE`, to represent decimal values. This approach ensures consistent precision across functions and avoids hard-coding a specific numeric scale like `1e18`. The documentation for the parameters and return value of `PeggedSwapMath.solve` currently states that values are scaled by `1e18`, but they should be scaled by `ONE` as defined in the library. This inconsistency could mislead future integrators.

Consider updating the `PeggedSwapMath.solve` NatSpec to state that inputs and outputs are scaled by `ONE`, and align wording with other functions that already reference `ONE` instead of `1e18`.

Update: *Resolved in pull request #94.*

N-05 Incorrect Error and Parameter Naming

The errors defined in the `Controls` instruction use incorrect naming parameters for the first `address` argument. They use `maker` instead of `taker` as the parameter name, whereas these errors are related to the `taker`. Moreover, in the `TakerTokenBalanceSupplyShareIsLessThatRequired` error name, 'That' should be replaced with 'Than'.

Consider updating the names of the aforementioned errors and their parameters to avoid confusion.

Update: *Resolved in pull request #89.*

N-06 Redundant Error Selector in `tryChopTakerArgs()`

In `tryChopTakerArgs()`, the `TryChopTakerArgsExcessiveLength` error selector passed to `data.slice()` can never trigger. The function clamps `length` to `data.length` using `Math.min()`, so `length <= data.length` is always `true`. The bound check in `slice()` verifies that `begin > calls.length`, which is never satisfied. This makes the error selector dead code and adds unnecessary gas cost.

Consider replacing the `data.slice(length, TryChopTakerArgsExcessiveLength.selector)` with `data.slice(length)` to use the non-bound-checking variant, since the clamping already ensures bound safety.

Update: *Resolved in pull request #90.*

N-07 Unused Imports

The `import { ITakerCallbacks } from "../interfaces/ITakerCallbacks.sol";` import in `TakerTraits.sol` is unused.

Consider removing unused imports to improve the overall clarity and maintainability of the codebase.

Update: *Resolved in pull request #91.*

N-08 Unused Errors

Throughout the codebase, multiple instances of unused errors were identified:

- The `DecayApplySwapAmountsRequiresAmountsToBeComputed` error in `Decay.sol`
- The `MakerTraitsMissingProgramData` error in `MakerTraits.sol`
- The `TakerTraitsMissingHookTarget` error in `TakerTraits.sol`
- The `RunLoopSwapAmountsComputationMissing` error in `VM.sol`

To improve the overall clarity, intentionality, and readability of the codebase, consider either using or removing any currently unused errors.

Update: *Resolved in pull request #91.*

N-09 Missing Security Contact

Providing a specific security contact (such as an email address or ENS name) within a smart contract significantly simplifies the process for individuals to communicate if they identify a vulnerability in the code. This practice is quite beneficial as it permits the code owners to dictate the communication channel for vulnerability disclosure, eliminating the risk of miscommunication or failure to report due to a lack of knowledge on how to do so. In addition, if the contract incorporates third-party libraries and a bug surfaces in those, it becomes easier for their maintainers to contact the appropriate person about the problem and provide mitigation instructions.

Throughout the codebase, no security contact was identified.

Consider adding a NatSpec comment containing a security contact above each contract definition. Using the `@custom:security-contact` convention is recommended as it has been adopted by the OpenZeppelin Wizard and the ethereum-lists.

Update: *Resolved in pull request #97.*

N-10 Lack of Indexed Event Parameters

Within `SwapVM.sol`, the `Swapped` event does not have indexed parameters.

To improve the ability of off-chain services to search and filter for specific events, consider indexing event parameters.

Update: *Acknowledged, not resolved.*

N-11 Incomplete Docstrings or Missing Docstrings

Throughout the codebase, multiple instances of incomplete docstrings were identified:

- The `public` functions, function arguments, important structs or return values in `SwapVM`, `IMakerHooks`, ISwapVM, `ITakerCallbacks`
- The `MakerTraits` and `TakerTraits` libraries, and important functions such as `runLoop()` in the `Vm` library
- The `build`/`parse` functions and `public` view functions like `concentratedBalance` function
- The `opcodes` and `routers` contracts

Consider thoroughly documenting all functions/events (and their parameters or return values) that are part of a contract's public API. When writing docstrings, consider following the Ethereum Natural Specification Format (NatSpec).

Update: *Resolved in pull request #92.*

N-12 Missing Explicit Validation for `amountOut < balanceOut` in the `XYCSwap::_xycSwapXD` Instruction

In the `ExactOut` branch of the `_xycSwapXD` function, the `balanceOut - amountOut` formula is used as a denominator without an explicit check that `amountOut < balanceOut`. If `amountOut >= balanceOut`, the transaction reverts with an opaque panic (underflow or division-by-zero) rather than a descriptive error, making debugging and integration more difficult.

Consider adding an explicit `amountOut < balanceOut` check to avoid implicit underflow/division-by-zero reverts.

Update: *Acknowledged, not resolved. The team stated:*

> We strive to keep the SwapVM protocol as gas-efficient as possible, so we will not add this check in this case.

N-13 `Decay._decayXD` Can Underflow `balanceOut` When Decayed Offsets Exceed Current Reserves

The `Decay` instruction maintains per-position offsets in `_offsets[orderHash][token][direction]` and applies them before evaluating the remaining VM program. In `_decayXD`, `balanceIn` is increased and `balanceOut` is decreased by time-decayed values returned by `DecayingOffsetLib.getOffset`.

`balanceOut` is decremented via a checked subtraction (`ctx.swap.balanceOut -= ...`) without validating that the decayed offset is less than or equal to the current `balanceOut`. If `balanceOut` decreases independently while the stored offsets persist (for example, due to withdrawals or other strategy instructions), the subtraction underflows and reverts. This blocks `swap()` and `quote()` for that `orderHash` and direction until either `balanceOut` increases or the offset decays to a value below `balanceOut` (bounded by the `uint16`-encoded decay period, at most 65,535 seconds ~ 18 hours).

Consider clamping the applied offset (for example, `offset = min(offset, balanceOut)`) or using saturating subtraction so that `Decay` cannot revert due to underflow. Consider updating stored offsets when clamping occurs, or reverting with a dedicated error that clearly signals insufficient reserves for applying the current offset.

Update: *Acknowledged, not resolved.*

N-14 16-Bit Jump Targets Mismatch 256-Bit PC, Causing Unreachable Code and Truncated Jumps

The VM represents the program counter `ctx.vm.nextPC` as a `uint256` byte offset and `ContextLib.runLoop` only enforces `ctx.vm.nextPC < programBytes.length`, with

no cap on `programBytes.length`. `ProgramBuilder.build` similarly imposes no limit on total program size. As a result, programs larger than 65,535 bytes are valid and executable.

Control-flow opcodes in `Controls` encode/decode jump targets as `uint16`. Specifically, `ControlsArgsBuilder.buildJump` and `ControlsArgsBuilder.buildJumpIfToken` emit 2-byte targets, while `Controls._jump`, `Controls._jumpIfTokenIn`, and `Controls._jumpIfTokenOut` read `bytes2`, cast to `uint16`, and set `ctx.setNextPC`. Targets at byte offsets `>= 65536` are therefore unrepresentable via these opcodes, and any larger value is truncated to the low 16 bits, potentially causing silent misdirected execution and leaving intended code unreachable. Although `Extruction._extruction` can set an arbitrary `uint256 nextPC`, the built-in `Controls` jumps cannot.

Consider aligning addressing widths across the VM. Options include enforcing `programBytes.length <= type(uint16).max` (e.g., in `ContextLib.runLoop` or order validation), or widening jump argument encoding/decoding in `Controls` and `ControlsArgsBuilder` to at least `uint32`/`uint256`. In addition, consider validating jump targets during program construction and rejecting overflow/truncated targets, and document the invariant so program builders do not rely on unreachable regions.

Update: *Resolved in pull request #93.*

N-15 Dynamic Fee Provider Call Allows Malformed `returndata` to Trigger Generic ABI Revert

In `_dynamicProtocolFeeAmountInXD` and `_aquaDynamicProtocolFeeAmountInXD`, dynamic protocol fees are obtained via `staticcall` to `feeProvider` using `IProtocolFeeProvider.getFeeBpsAndRecipient`. The implementation checks the call's `success` flag and then decodes the return payload into `(uint32,address)` before applying the fee within the VM loop used by `swap(...)` and `quote(...)`.

The code treats `success == true` as proof of a well-formed return value, but this does not hold. If `result` is empty, short, or malformed, `abi.decode(result, (uint32,address))` reverts with a generic ABI decoding error, bypassing the intended `FeeProtocolProviderFailedCall()` revert. This is reachable when `feeProvider` is an EOA or a non-conforming contract that returns incorrect data, producing inconsistent error signaling and causing an operational DoS for orders that rely on dynamic fees in both `swap(...)` and `quote(...)`.

Consider hardening the decoding boundary by validating the return payload before decoding. Require `success` and `result.length == 64` (the ABI size for `(uint32,address)`), and revert with `FeeProtocolProviderFailedCall()` otherwise.

Update: *Resolved in pull request #87.*

N-16 Misdocumented `XYCConcentrate XD/2D` Argument Layout

The VM forwards raw `args` bytes to opcode implementations via `ContextLib.runLoop` without schema enforcement. For XD instructions, `parseXD` expects `tokensCount`, `tokens[]`, `deltas[]`, then a trailing `liquidity` word, and `buildXD` encodes exactly that. However, `_xycConcentrateGrowLiquidityXD` docstrings describe the third segment as `args.initialBalances[]` and omit the required `liquidity`. Similarly, `_xycConcentrateGrowLiquidity2D` omits documenting the trailing `liquidity` required by `parse2D`.

Consider aligning documentation and code by updating the docstrings in `_xycConcentrateGrowLiquidityXD` and `_xycConcentrateGrowLiquidity2D` to explicitly specify `deltas` and a final `liquidity` word, matching `XYCConcentrateArgsBuilder.parseXD` / `parse2D`.

Update: *Resolved in pull request #82.*

N-17 Division-By-Zero in `computeDeltas` due to `sqrt` Rounding Down and Missing Zero Validation

The `computeDeltas` library function computes `deltaA`, `deltaB`, and `liquidity` based on balances and price bounds. It derives the denominators from `sqrtPriceMin` and `sqrtPriceMax`, and then divides by `sqrtPriceMin - ONE` and `sqrtPriceMax - ONE`. The only input validation is `priceMin <= price && price <= priceMax`.

However, `Math.sqrt` floors its result, which allows `sqrtPriceMin == ONE` when `price` is slightly greater than `priceMin`, and `sqrtPriceMax == ONE` when `price` is slightly less than `priceMax`. In these cases, the branches `price == priceMin` and `price == priceMax` are false, but the denominators become zero, causing `deltaA` or `deltaB` to revert with a division-by-zero panic. In addition, the function does not enforce `priceMin != 0` or `price != 0`, permitting zero inputs that trigger division-by-zero in `price * ONE /`

`priceMin` or `priceMax * ONE / price` before the ternaries apply. The result is an unexpected DoS on near-boundary inputs and inconsistent error signaling.

Consider hardening input checks by requiring `priceMin > 0` and `price > 0`, and by asserting non-zero denominators (e.g., `require((price == priceMin) || (sqrtPriceMin > ONE))` and `require((price == priceMax) || (sqrtPriceMax > ONE))`). In addition, consider treating rounded-equal cases as equal by setting `deltaA = 0` when `sqrtPriceMin <= ONE` and `deltaB = 0` when `sqrtPriceMax <= ONE`, or reverting with a domain-specific error to maintain consistency.

Update: *Resolved in pull request #82. The team stated:*

> The latest version of XYCConcentrate is stateless and doesn't provide computeDeltas helper function.

N-18 Systematic Rounding Down in Grow Liquidity Causes Price-Range Drift

The liquidity growth flow of the `XYCConcentrate` contract adjusts in-memory balances via `concentratedBalance` using `liquidity[orderHash]` and `initialLiquidity`, and then persists a new scale in `_updateScales` function based on the post-swap invariant. This loop is executed in both `_xycConcentrateGrowLiquidity2D` and `_xycConcentrateGrowLiquidityXD`.

The observed issue stems from floor rounding in two points: `delta * currentLiquidity / initialLiquidity` inside `concentratedBalance`, and `Math.sqrt(newInv)` inside `_updateScales` where `newInv = (ctx.swap.balanceIn + ctx.swap.amountIn) * (ctx.swap.balanceOut - ctx.swap.amountOut)`. These quantizations introduce a systematic loss of precision that accumulates across iterations, shifting the implied bounds and altering quotes near the edges of the target range.

Consider eliminating systematic rounding bias in the grow-liquidity loop by using a full-precision multiply-divide with explicit rounding (e.g., `Math.mulDiv(delta, currentLiquidity, initialLiquidity, Math.Rounding.Up)`), after confirming the correct rounding direction for `balanceIn` and `balanceOut`. If this is not addressed in code, consider documenting the rounding behavior and its maximum cumulative impact so that integrators can account for potential price-range drift.

Update: *Resolved in pull request #82.*

N-19 Concentrated-Liquidity Docs are Outdated

The example on concentrated-liquidity in `README.md` file uses a non-existent `computeDeltas` function and a four-argument `build2D` function. The current `build2D` implementation takes only `sqrtPriceMin` and `sqrtPriceMax` as input arguments. Integrators following the `README` will get wrong or failing code as a result.

Consider updating the `README.md` to the current logic implementation of the `XYCConcentrate.sol` contract.

Update: *Resolved in pull request #105.*

N-20 Missing Explicit Validation of `sqrtPmin` < `sqrtPmax` During Liquidity Computation

In `XYCConcentrate` instruction, neither `computeLiquidityFromAmounts` nor `computeBalances` validates that `sqrtPmin < sqrtPmax`. If called with `sqrtPmin >= sqrtPmax`, the functions revert with an opaque arithmetic underflow (`Panic`) rather than a descriptive error. While `build2D` validates this at bytecode construction time, these library helpers can be called independently.

Consider verifying `sqrtPmin < sqrtPmax` with a descriptive custom error at the top of `computeLiquidityFromAmounts` and `computeBalances` functions.

Update: *Resolved in pull request #105.*

Recommendations

Strengthening Documentation for Makers and Takers

Several instructions are interdependent and manage their own internal storage state. Some instructions also rely on explicit assumptions about preceding or subsequent instructions, or on guarantees provided by the maker. As such, it is recommended to provide **guide-style documentation for makers** that explains how to construct efficient and safe programs, clearly outlining the assumptions, expectations, and constraints associated with each instruction. This would help prevent the creation of unintended or harmful programs.

Similarly, it is recommended to maintain **dedicated documentation for takers** in order to help them identify potentially malicious programs authored by makers and to reason about the underlying trust assumptions. For example, the presence of an `extruction` instruction in a program effectively requires the taker to place full trust in the maker, a risk that should be clearly documented and easy to recognize.

Conclusion

The audit identified 3 critical-, 2 high-, and 12 medium-severity issues, along with multiple low-severity and informational findings, which provide a clear path forward before the production launch. In order to strengthen the security of the codebase, it is recommended to not only fix the reported issues but also improve user-facing documentation, strengthen the end-to-end test suite (particularly around fee-related instructions), and add test suites specifically designed to ensure that these vulnerabilities do not re-emerge.

The recommended fixes are self-contained and do not significantly alter the protocol design. However, any scope expansion, especially involving multi-token integration or logical modifications, should undergo a new audit to guarantee safe and secure interactions for all supported tokens.

The most severe findings include invariant-breaking edge cases in `PeggedSwap` that can lead to materially incorrect pricing and reserve loss under certain parameterizations or swap directions, and an XYCConcentrate multi-token accounting flaw where shared liquidity tracking across pairs enables cyclic arbitrage. Several major issues were also concentrated in the fee implementation and its interaction with nested `runLoop()` wrappers, creating order-dependent fee behavior and liveness hazards.

Addressing the **critical-, high-, and medium-severity** issues should be treated as a high priority, alongside strengthening parameter constraints and expanding tests for adversarial opcode compositions and long-term economic invariants to prevent regressions. Given the interdependencies between certain instructions, a recommendation is made to maintain strong documentation describing each instruction's capabilities, constraints, and trust assumptions, to help makers build correct and efficient programs.

The 1inch team is commended for being highly cooperative and providing clear explanations throughout the audit process.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.